Mini Task Management System

REST API Documentation

Base URL

http://localhost:8080/api

Authentication APIs

1. Register User

Endpoint : POST /auth/register

Description : Creates a new user account.

Headers

Header	Value
Content-Type	application/json

Request Body

```
{
  "email": "user@test.com",
  "password": "123456",
  "role": "USER"
}
```

Response

```
{
  "message": "User registered successfully"
}
```

Status Codes

Code	Description
201	User created
400	Invalid input

2. Login User

Endpoint : POST /auth/login

Description : Authenticates user and returns JWT token.

Headers

Header	Value
Content-Type	application/json

Request Body

```
{
  "email": "user@test.com",
  "password": "123456"
}
```

Response

```
{
  "token": "JWT_TOKEN"
}
```

Status Codes

Code	Description
200	Login successful
401	Invalid credentials

Task APIs

All task APIs require authentication.

Header

Authorization: Bearer JWT_TOKEN

3. Create Task

Endpoint : POST /tasks

Request Body

```
{
  "title": "Finish Assignment",
  "description": "Complete API documentation",
  "status": "TODO",
  "priority": "HIGH",
  "dueDate": "2026-03-20"
}
```

Response

```
{
  "id": 1,
  "title": "Finish Assignment",
  "description": "Complete API documentation",
  "status": "TODO",
  "priority": "HIGH",
  "dueDate": "2026-03-20"
}
```

Status Codes

Code	Description
201	Task created
400	Invalid request

4. Get Tasks

Endpoint : GET /tasks

Description : Returns tasks for the logged-in user. And ADMIN users can view all tasks.

Response

```
[
 {
  "id": 1,
  "title": "Finish Assignment",
  "description": "Complete documentation",
  "status": "TODO",
  "priority": "HIGH",
  "dueDate": "2026-03-20"
 }
]
```

5. Update Task

Endpoint : PUT /tasks/{id}

Example : PUT /tasks/1

Request Body

```
{
  "title": "Update Assignment",
  "description": "Fix API documentation",
  "status": "IN_PROGRESS",
  "priority": "MEDIUM",
  "dueDate": "2026-03-22"
}
```

Response

```
{
 "message": "Task updated successfully"
}
```

6. Delete Task

Endpoint : DELETE /tasks/{id}

Example : DELETE /tasks/1

Response

```
{
 "message": "Task deleted successfully"
}
```

Task Object Structure

Field	Type	Description
id	Long	Task ID
title	String	Task title
description	String	Task details
status	Enum	TODO, IN_PROGRESS, DONE
priority	Enum	LOW, MEDIUM, HIGH
dueDate	Date	Task deadline
createdAt	Timestamp	Creation time
updatedAt	Timestamp	Update time

HTTP Status Codes

Code	Meaning
200	Successful request
201	Resource created
400	Bad request
401	Unauthorized
403	Forbidden
404	Resource not found
500	Internal server error